Zigで覗く、Linuxの非同期I/O

epollからio_uringまで

いつも使う async/await の、少し下へ。

async/awaitの下では何が起きている？

```
const response = await fetch(url);
const text = await response.text();
```

JavaScript / Python
async/await
→
Runtime
実行と再開を管理
→
Event Loop
I/O待機
→
epoll / kqueue
Kernelの機構を利用

ネットワークI/Oの実装例。言語・runtime・OS・操作によって経路は異なる。

普段使う非同期処理の下には、OSのI/O機構がある

最近よく聞く io_uring

PostgreSQL 18	新しい非同期I/O基盤。Linuxで `io_method = io_uring` を選択可能。
Dragonfly	Redis / Memcached互換データストア。LinuxのI/OにIo_uringを活用。
Node.js / libuv	ファイルI/Oへの採用の経緯。libuv 1.49以降は既定をthread poolへ戻した。

PostgreSQL 18の公式発表は新AIO全体で「一部の条件で最大3倍」。io_uring単独の効果ではない。

なぜ、これほどいろいろな技術で注目されているのか？

ところで、Zigとは？

Cに近い低レイヤーを扱える
システムプログラミング言語。

明示的に扱う設計

メモリの確保や解放、
リソースの寿命をコードで管理する。

OSの近くを触りやすい

クロスコンパイルとCとの相互運用。
Linux syscallを呼ぶ標準ライブラリもある。

今回はZigを使って、Linuxの仕組みを実際に観察してみる

Zigでシステムコールを試す

```
const n = std.os.linux.write(1, "hello\n", 6);
```

Zig 0.17.0 / Linux aarch64で実測

```
$ strace -e write ./raw_write
write(1, "hello\n", 6) = 6
```

1　fd：出力先　　**"hello\n"　データ**　　**6　要求byte数**　　**= 6 / n　書けたbyte数**

「標準出力へ6 byte書く」が、OSへの依頼として見えた

システムコールは、OSへの依頼



同じ関数呼び出しに見えても、ここではカーネルへ処理を依頼する

fdは、プロセスが持つ番号

このプロセスのfdテーブル

fd	接続先の例
0	標準入力 stdin
1	標準出力 stdout
2	標準エラー stderr
3	ファイル hello.txt
4	socket / pipe

```
const n = linux.read(0, &buf, buf.len);
```

0：どこから
buf：どこへ
n：実際に読めたbyte数

read / writeは行単位ではなく、
byte列を扱う。

fdはファイル専用ではない。socketやpipeも同じ番号の仕組みで扱う

データがまだないとき、どうする？

Blocking

read(fd)

↓

データが来るまで待つ

↓

結果を返す

呼び出したスレッドは先へ進まない。
待機中にCPUを使い続ける、という意味ではない。

Non-blocking

read(fd)

↓

データがない

↓

EAGAIN：今は読めない

EAGAIN自身は再開通知を予約しない。
即座に再試行し続けるとbusy loopになる。

たくさんのI/Oを効率よく待つには、別の仕組みが必要になる

なぜepollは低Ready率に強い？

1,024 fdのうち、**読み取り可能なのは10 fd**なら？

poll：毎回fd集合を走査する

1,024 fdの配列	→	全体を確認する	→	Readyな10 fd

epoll：監視対象を先に登録する

1,024 fdを事前登録	→	通知でReady Listへ	→	10 fdを確認・取得

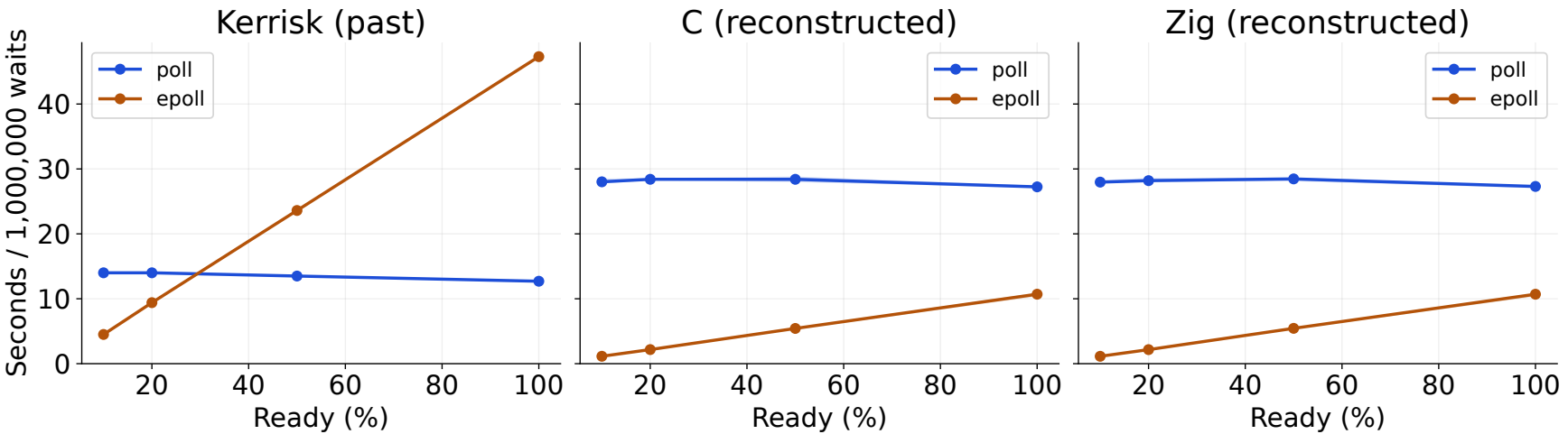
Wait Queue（通知の待ち先） → `ep_poll_callback()` → Ready List

---

epollは通知を利用し、待機のたびに全fdを確認する仕事を減らす

Ready率 = 読み取り可能な監視fd数 / 監視fd総数。どちらも受信処理はアプリ側で行う。

Kerrisk氏の測定を、CとZigで追試



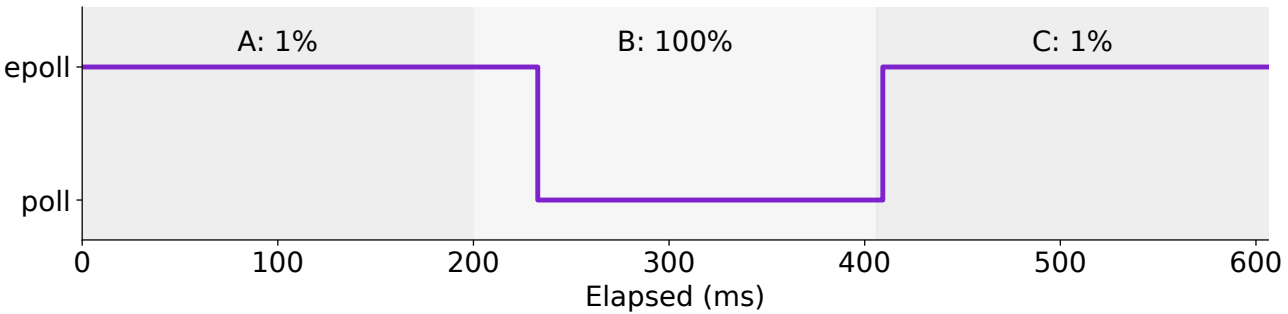
過去は高Ready率で**poll優位**。今回の条件ではepoll優位。

元Cは未入手。公開仕様から再構成し、過去と現在を別の実験として比較した。

I/O APIの性能は、何をどこまで測るかによっても変わる

今回：1,000 pipe fd・低位連続・100万回×各3回。timeout=-1、容量N、走査なしは仮定。OrbStack / Zig 0.17.0。

動的に切り替えれば速くなる？



Actual diagnostic run, seed 20261101 | 4096 fds, 1 KiB | separate from performance samples

各200 ms：系列間でも小差が変動

今回の中央値-0.19%。過去は改善未確認。

切り替えは中央値2回・計2.18 ms。

高Readyが長く続くと？

1秒：**+2.53%** ／ 5秒：**+4.51%**

固定モデルのまま、改善を確認した。

切り替える利益が、追随の遅れや切り替え費用を上回るか？

4,096 fd / 1 KiB / CPU 2 / 各10回のペア比中央値。A/C各200 ms、Bのみ延長。性能測定は診断無効。

io_uringは、I/Oを依頼して完了を受け取る

APPLICATION

read(fd) を記述

SQE：依頼の1件分

→

SUBMISSION QUEUE

SQ：依頼のキュー

SQEを投入する

→

KERNEL

LinuxがI/Oを処理

完了結果をCQへ

Applicationが結果を取得 ← **CQE：res = 128 byte** ← **CQ：完了のキュー**

SQ = Submission Queue
SQE = Submission Queue Entry

CQ = Completion Queue
CQE = Completion Queue Entry

「読めそうか？」を聞く代わりに、readの結果を受け取れる

なぜio_uringは速くなり得る？

まとめて依頼	複数のI/Oをバッチで投入し、同時に進行させる。
共有リングで受け渡し	依頼・結果の受け渡しを効率化。syscall境界をまたぐ回数を減らせる。
繰り返しの準備を減らす	registered buffers：バッファを事前登録し、操作ごとの準備を削減。

I/Oの待ち時間と、1件ごとの手間をまとめて減らす余地がある

syscallが不要になるわけではない。効果は操作・設定・ワークロードに依存する。

epollは「I/Oできる状態」を教える

epoll_wait()	→	fd 5 がread可能	→	read(fd5)	→	データを取得
登録したfdをまとめて待つ		準備完了の通知		アプリがI/Oを実行		戻り値を確認

Reactorモデル：

準備完了を知ってから、自分でI/Oする。

epoll自身がreadを代行するわけではない

Edge-triggeredではnon-blockingで処理し、EAGAINで再待機。EOF・エラーも判定する。

「準備完了」と「操作完了」の違い

epoll / Reactor

準備完了を待つ

↓

アプリがreadを実行

↓

結果を受け取る

socketの待機で広く利用。
通知後のI/Oと再試行はアプリが管理。

io_uring / Proactor的な利用

readを登録する

↓

操作の完了を待つ

↓

結果を受け取る

操作の投入と完了をまとめて扱える。
完了までのバッファ寿命などの管理が必要。

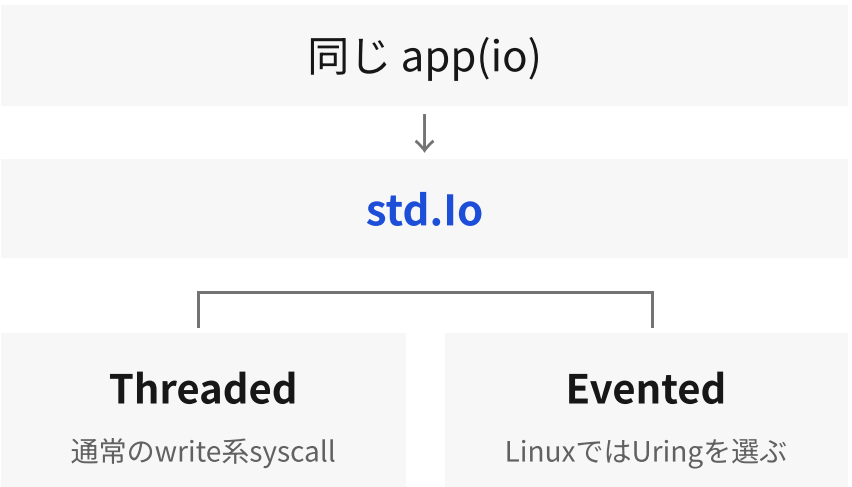
同じreadでも、アプリとOSの役割の分け方が違う

io_uringにもpollによるreadiness通知がある。完全に排他的なモデルではない。

Zigのstd.Ioで、I/O実装を分離する

```zig
fn app(io: std.Io) !void {
    try std.Io.File.stdout().writeStreamingAll(
        io,
        "Hello, World!\n",
    );
}
```

ioを受け取り、**同じ文字列**を出力する。



切り替えるのは呼び出し側の初期化。
Eventedは実験的で、0.17.0ではビルドに制約あり。

アプリの処理を保ったまま、I/Oの実装を選ぶ設計

io_uringや実装の差し替えはZig固有ではない。今回は標準ライブラリの設計を見る。

同じコードで、syscallは変わる？

Threaded：実行・strace確認済み

```
writev(1,
 [{iov_base="Hello, World!\n",
   iov_len=14}], 1) = 14
```

writevで**stdout（fd 1）**へ
14 byteを出力。

Evented：ビルド時に停止

```
error: no field named
'processReplacePath'
in struct 'Io.VTable'
```

0.17.0の標準ライブラリ内の不整合。
Evented経由のsyscallは未観測。

低レベルのstd.os.linux.IoUringは別途実行済み。
straceでsetup / enterの成功を確認。

差し替える設計は確認できた。両者のsyscall比較は未完了

Linux aarch64 / Zig 0.17.0・標準ライブラリ未改変。straceはPIDを除き折り返し。性能比較なし。

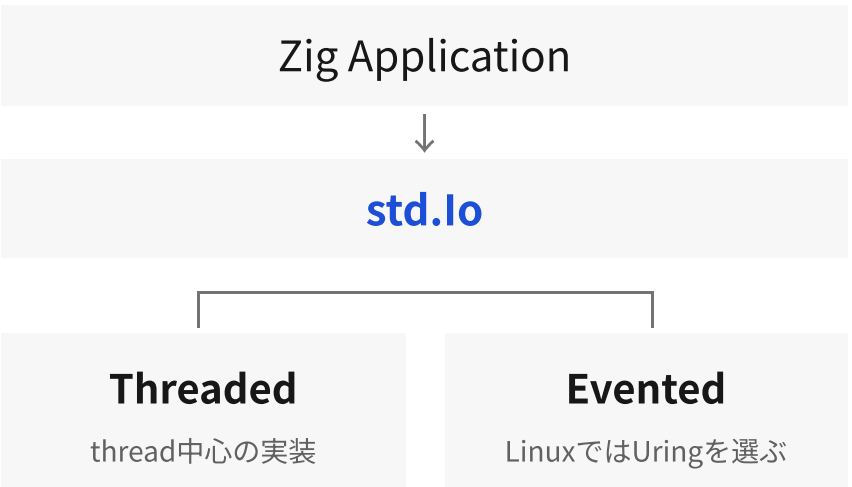
std.Ioにも、実装ごとの対応差がある

Threaded

既定のstd.process.Initが渡す実装。
今回の出力処理は実行できた。

Evented / Uring

Linux向けの実験的な実装。
0.17.0はビルド不整合があり、
listen / accept / connect等にも未対応。

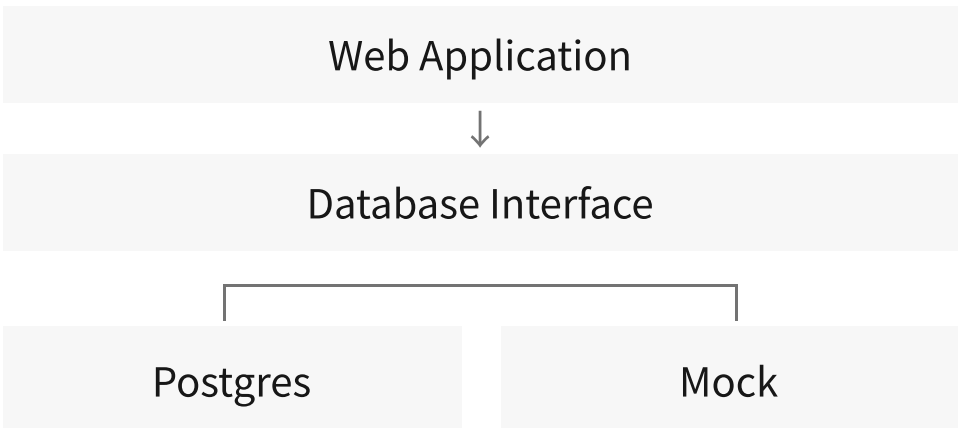


インターフェースが共通でも、対応状況は確かめる

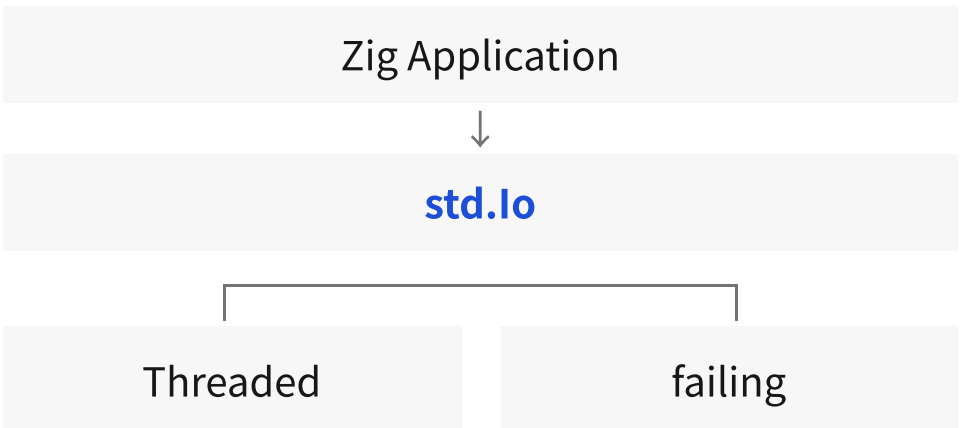
Futureで関数呼び出しも扱えるが、並行実行の可否やキャンセルの動作も実装に依存する。

I/Oを差し替えられると、何がうれしい？

Webでよく見るDI



ZigのI/Oインターフェース



同じapp(io)を実行　　Threaded：**helloを出力**　　failing：**error.InputOutput**

実装への依存を局所化し、同じ処理の成功・失敗を確かめられる

実測はこの小さな出力処理のみ。failingは汎用のMockや全動作を再現する実装ではない。

io_uringなら、何でも速い？

速くなる条件がある。

手間が上回ることもある

少量のI/Oでは、リングや要求管理の
オーバーヘッドを回収できない場合がある。

使う側の設計も変わる

並行する要求、バッファの寿命、
キャンセルを扱う必要がある。

ボトルネックはどこか

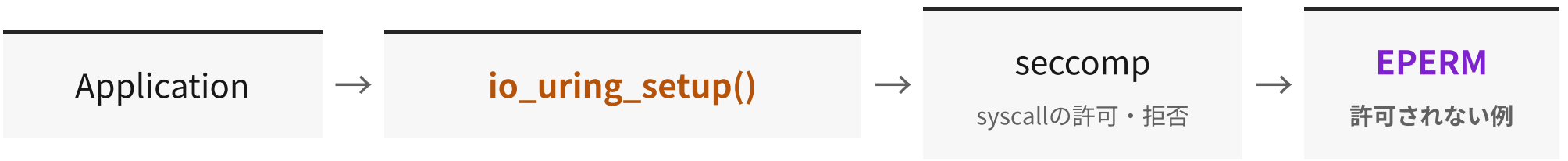
CPU側の手間を減らしても、ストレージや
ネットワークの限界はそのまま残る。

環境によって違う

Linuxのバージョン、対応操作、
runtimeの実装を確かめる。

ワークロードを決めて測る。epollが合う場面もある

使いたくても使えないことがある



Dockerの既定プロファイルの例

公式資料ではio_uring関連syscallをブロック。
実際の挙動はEngine・適用プロファイルで変わる。

別のI/O方式を用意する

初期化の可否を確認し、
thread poolなどへのfallbackを検討する。

性能だけでなく、実行環境のポリシーも採用条件になる

2026-10-18時点のDocker公式資料。EPERMの原因はseccompだけとは限らない。

非同期I/Oの裏側を覗いてみた

01 async/awaitの下には、OSのI/O機構がある。

02 io_uringは、I/Oの依頼と完了を効率よく扱うLinuxの仕組み。

03 epollとio_uringには、異なる設計と得意分野がある。

04 Zigでは、syscallの観察もstd.Ioによる抽象化もできる。

05 性能・互換性・セキュリティを確かめて使う。

普段の非同期処理。その少し下に、Linuxの面白い仕組みがある。

参考資料

Zig / runtime

- Zig 0.17.0 release notes / 標準ライブラリ
- Zig overview
- libuv: design / filesystem
- Python: event loop

Linux

- syscalls(2) / read(2) / write(2)
- poll(2) / epoll_wait(2)
- epoll(7) / io_uring(7)
- io_uring_register(2)
- Linux v7.0.14: eventpoll.c
- Kerrisk: epoll講義

採用事例 / 運用

- PostgreSQL 18 release
- PostgreSQL 18: io_method
- Dragonfly: architecture
- Dragonfly: SSD tiering
- Docker: seccomp
- Kernel: FUSE-over-io_uring